

Apuntes Semana 09 - Clase del Jueves

Mario Adrián Sánchez Ponce, 2022167079, Instituto Tecnológico de Costa Rica

I. PERCEPTRÓN

Término creado por Frank Rosenblatt. Consiste en una regresión logística donde la función de pérdida se trata de la "Hinge Loss". Este término se planteó desde los años 70 con la idea de que resolvería tareas al reprogramar un solo perceptrón a la tarea. El problema es que este solo resuelve problemas lineales. Siendo incapaz de resolver un XOR, un problema considerado sencillo, por su no linealidad.

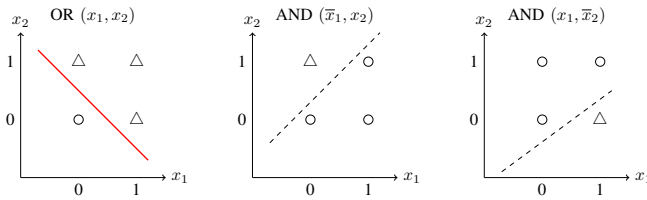


Figura 1. Representación gráfica de predicción de compuertas lógicas.

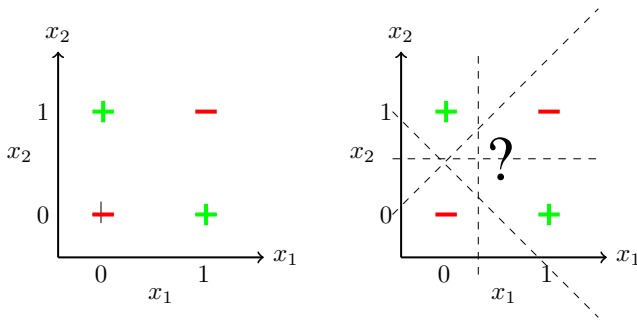


Figura 2. Representación del problema XOR y su no separabilidad lineal.

Como se puede analizar, no se puede resolver con perceptrón solamente, pues se necesita varias líneas para determinar si es valor positivo o negativo. Sin embargo, las redes neuronales si son capaces de resolverlo pues es capaz de atacar problemas no lineales.

II. INSPIRACIÓN BIOLÓGICA

El término de redes neuronales viene del sistema de comunicación neuronal donde las neuronas se encuentran conectadas entre sí. Esto por medio de las dendritas que contienen las entradas de la información. El núcleo recibe la información y la envía a la siguiente neurona a la que está conectada, decidiendo como va a pasar y que es lo que se va a enviar. En el campo de la inteligencia artificial actúa de manera similar, donde se pasa la función lineal y por medio de una función de activación se procede a un output único.

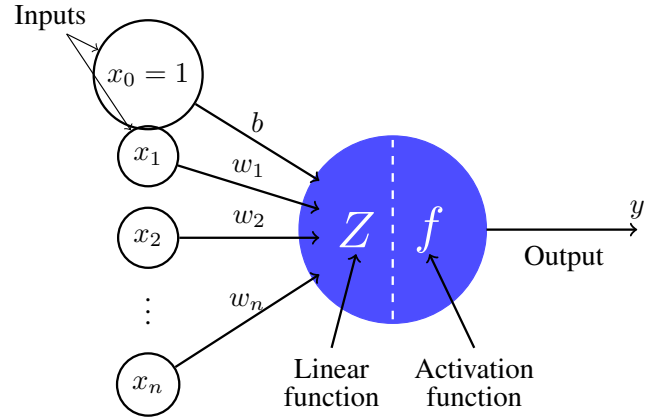


Figura 3. Ejemplificación de una red neuronal

III. FUNCIÓN DE ACTIVACIÓN

La neurona (regresión logística) se compone por una función lineal y no lineal, la función de activación se refiere a la no lineal, que se encarga de agregar

III-A. Función Lineal

Funciona con respecto a problemas simples. En cuanto a los problemas, la derivada no aprende de la entrada, esto ya que la gradiente siempre será el mismo en cada iteración.

$$y = mx + b \quad (1)$$

$$\frac{d}{dx}(mx + b) = m \quad (2)$$

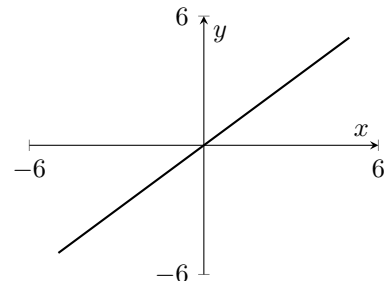


Figura 4. Función lineal. Si lo amerita

III-B. Función sigmoide

Permite la activación entre valores de 0 y 1. Estrictamente creciente y positiva y es acotada. La derivada tiene un comportamiento que provoca que haya puntos muertos lo que

significa que el gradiente se desvanece en los extremos de la función sigmoide y es muy cercana a 0. Esto hace que multiplique por 0 todos los resultados, haciendo ineficiente el entrenamiento, estancándolo.

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (3)$$

$$\frac{d\sigma(x)}{dx} = \sigma(x) (1 - \sigma(x)) \quad (4)$$

...

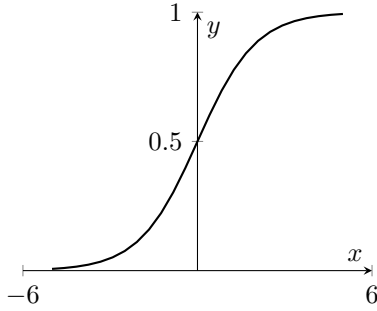


Figura 5. Función sigmoide.

III-C. Función tangente hiperbólica

Acortada a tanh, se activa entre -1 y 1, estrictamente creciente similar a la sigmoide. Se utilizaba mucho en modelos primitivos LSTM. La derivada sufre del mismo problema que la función sigmoide aunque el gradiente avanza un poco más sobre la función de costo.

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} \quad (5)$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x) \quad (6)$$

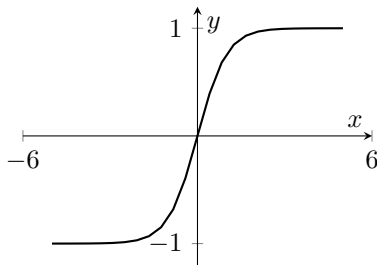


Figura 6. Función tangente hiperbólica. Los valores se acotan en [-1, 1]

III-D. Función ReLU

Función creada con el propósito de reemplazar la función sigmoide y tanh dentro de las capas ocultas en Deep Learning. Se acota por debajo de cero pero no sobre los positivos. Estrictamente creciente. Mata las activaciones de las neuronas lo cual es un problema.

$$\text{ReLU}(x) = \max(0, x) \quad (7)$$

$$\frac{d}{dx} \text{ReLU}(x) = \begin{cases} 0, & x < 0 \\ 1, & x > 0 \end{cases} \quad (8)$$

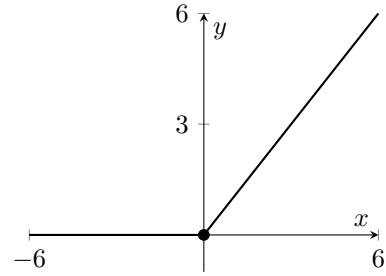


Figura 7. Función ReLU

III-E. Otras funciones de activación

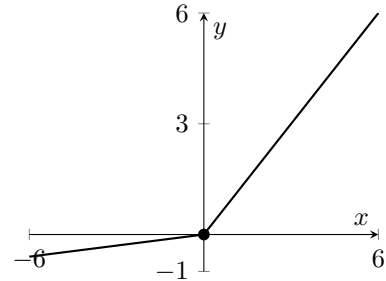


Figura 8. Función Leaky ReLU. Permite que la parte negativa tenga una pendiente

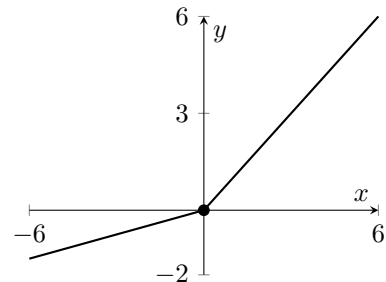


Figura 9. Función Parametric ReLU.

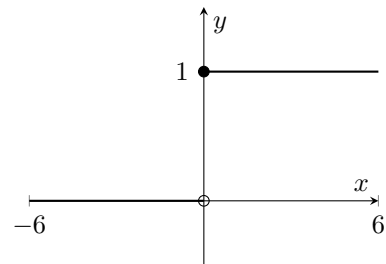


Figura 10. Función Binary Step.

La selección de la función depende tanto del problema y de la arquitectura presentada para la solución. Por ejemplo, si la

salida es acotada de 0 a 1, se utiliza sigmoide, si es de mayor a 0 se utiliza ReLU y así con cada situación. También se debe tomar en cuenta cada capa intermedia con su propia función de activación que pueden ser diferentes a la de salida. Todas las neuronas de la capa utilizan la misma función de activación

IV. PERCEPTRÓN MULTICAPA (MLP)

En esta arquitectura, todas las neuronas están conectadas con las de la siguiente capa hasta la capa de salida. La notación de los pesos varían dependiendo del libro pero el significado es el mismo siendo $W = \Theta$. Retomando lo de las clases pasadas, en el sistema MNIST se utiliza esto con la función sigmoide. Para esto se necesita primero computar la transformación lineal de los pesos. Se toman los resultados de cada una de las 10 neuronas y se aplica la función de activación

$$h^{(0)} = \text{sigmoid} \left(XW^{(0)} + b^{(0)} \right)$$

Para la próxima capa se necesita los resultados de la capa anterior y se aplica la nueva función de activación. Esto con cada capa formando una anidación de funciones hasta llegar a la capa de salida.

$$h^{(1)} = \text{sigmoid} \left(h^{(0)}W^{(1)} + b^{(1)} \right)$$

$$h^{(n)} = \text{sigmoid} \left(h^{(n-1)}W^{(n)} + b^{(n)} \right)$$

No siempre es óptimo el uso de la función sigmoide. En las capas intermedias no es la función de activación más utilizada, en cambio se usa una de las mostradas en las figuras anteriores.

IV-A. Salida independiente

En la capa de salida con sigmoide, las probabilidades obtenidas son independientes donde la suma no es necesariamente 1. Esto significa que no hay una distribución de probabilidad y es propenso a que reconozca a varios datos. La idea es buscar una distribución para poder utilizar el One-Hot Vector y determinar la clase correctamente.

Por ello, la capa de salida es una función no lineal que no necesariamente es una función sigmonide.

Cuadro I

SALIDA DEL MODELO EN UNA DISTRIBUCIÓN Y VECTOR ONE-HOT

Output Layer	One-hot vector
0.05	0
0.05	0
0.06	0
0.04	0
0.1	0
0.5	1
0.05	0
0.05	0
0.05	0
0.05	0

IV-B. Función de costo

Función que permite evaluar la distribución final de la capa de salida. Es necesario una función para toda la capa. Asumiendo que se tiene la función de costo, se optimiza con el descenso de gradiente, calculando la derivada parcial de costo por cada parámetro, es decir para cada neurona de cada capa y cuanto contribuyen a la función L.

$$\frac{\partial L}{\partial w_{ij}} \quad (9)$$

Donde J es la capa e i es la neurona.

Es un proceso iterativo donde se necesitan todas las derivadas parciales. Se hace el cálculo para determinar que tan correcto es la prueba

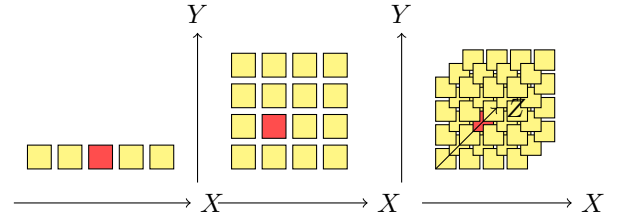
V. ¿CUÁNTAS CAPAS SE USAN EN CADA NEURONA?

Se basa más en experimentación que en una ciencia exacta. Se busca que haya un balance sin overfitting o underfitting. Existe una jerarquía por capas. La primera se encarga de reconocer patrones y a medida que se acerca a la capa de salida se vuelve más específico en su reconocimiento. Por ejemplo

Líneas -> Figuras -> Números

VI. MADDDICIÓN DE DIMENSIONALIDAD

A medida de que haya un modelo más complejo, es más difícil de optimizar.



Intentar buscar un elemento se vuelve más complejo en cuanto se añadan dimensiones. Nuevos features añade complejidad, provocando entrenamientos más pesados y genera mayor computabilidad. Se busca la función más sencilla posible que resuelva el problema. De lo más básico a la más compleja. Está el Análisis de de componentes principales (PCA) que reduce la dimensionalidad, con eso se transforma los componentes a datos numéricos y le da variabilidad de los componentes principales y reduce el algoritmo en gran medida. .

VII. COMPORTAMIENTO JERÁRQUICO

De la misma manera que los humanos empiezan aprendiendo desde cosas simples hasta problemas complejos. Asi mismo funciona los modelos de redes neuronales, desde regresión lineal hasta MLP. Esto permite una ganancia exponencial en funciones como lo polinomios y la composición de funciones simples para otras de orden superior.

VII-A. Representación compacta

Con pocos pesos se puede modelar funciones complejas

VII-B. Convolutional Neural Networks

Más eficientes que una red neuronal simple, siendo un kernel que observa una fracción de la imagen y va obteniendo la información de esta. En el ejemplo pasa de low level, mid level y high level features. Esto ayudó a la modelación de inteligencia artificial para el lenguaje, donde se construyó un vector que nos permite ubicar una frase en un punto que está distribuido espacio vectorial. Donde los artículos se agrupan y lenguajes distintos como el chino salen de la zona del lenguaje. Esto se conoce como embeddings y ayuda al día de hoy en la comparación, donde transforma las frases en embeddings para analizar que tan cercanas son estas.

En pocas palabras, permite representar al mundo de forma eficiente. Es una composición basada en jerarquía de representaciones aprendidas y no predefinidas.